

System Design for Nicholas Vanderschantz

Kei Hea a Nic? Where is Nic?: Dynamic Digital Office Display System

Contact Information

cole.brewerton@gmail.com

williamsjamie24@gmail.com

fernandohimendra@gmail.com

navrajmann138@gmail.com

Table of Contents

1. Introduction 3

 1.1 Purpose 3

 1.2 Scope..... 3

 1.3 Overview 4

2. SYSTEM OVERVIEW 4

 2.1 Stakeholder Concerns 4

 2.2 Deployment Diagram 5

 2.3 Component Diagram 6

 2.4 Data Flow Diagram..... 7

 2.5 Level 1 Dataflow Diagram 8

 2.6 Requirements Matrix..... 10



1. Introduction

1.1 Purpose

This System Design Document (SDD) describes the architecture and design of the "Kei Hea a Nic? Where is Nic?" dynamic digital office display system, developed for Nicholas Vanderschantz and the School of Computing and Mathematical Sciences (SCMS) at the University of Waikato. This document builds on the requirements defined in the System Requirements Specification (SRS) and translates those requirements into a concrete system architecture, showing how the Diary Owner Web Application, Diary Server, and Digital Office Display will be structured, how they interact, and how data moves between them.

The intended audience for this document is:

- The Development Team: To guide implementation decisions and ensure the system built during the prototyping phase matches the agreed design.
- The Client (Nicholas Vanderschantz/SCMS): to confirm that the proposed design meets the needs identified in the requirements phase before development begins.
- Future Maintainers: to provide a reference for anyone extending or maintaining the system beyond the prototype stage.

1.2 Scope

The "Kei Hea a Nic?" system replaces the current practice of printed paper timetables outside academic office doors with a dynamic, digitally updated display. The system is made up of three integrated components: a web-based Diary Owner Application that allows academic staff to manage their availability, calendar, and contact details, a Diary Server that authenticates users and stores and distributes this information, and a Digital Office Display, running on a Raspberry Pi, that presents this information to visitors outside the academic's office in real time.

The goal of this design is to produce a working prototype that demonstrates the feasibility of a fully digital, self-updating availability system rather than a production-ready deployment. The objectives of the design are to ensure the three components can communicate reliably over HTTPS, that changes made by an academic update to the display within approximately 15 seconds, and that the system remains usable for a broad range of stakeholders, including staff with limited technical experience and visitors unfamiliar with the system.

The benefits of this design, once implemented, include more accurate and timely availability information for students and staff, reduced administrative effort for academics who no longer need to reprint schedules, and a more accessible and professional presentation of information across the school.

1.3 Overview

This document is organised into two main sections following the "Changes in Requirements" note. The Introduction (Section 1) outlines the purpose, scope and structure of the document. The System Overview (Section 2) forms the main body of the SDD and details the architecture of the system, including the stakeholders and their design concerns (2.1), the deployment diagram showing the hardware and software layout (2.2), the component diagram describing the main system modules (2.3), the data flow diagram showing how information moves through the system (2.4), an additional design model (2.5), and a requirements traceability matrix linking the design back to the SRS (2.6).

2. SYSTEM OVERVIEW

2.1 Stakeholder Concerns

Several groups of stakeholders have an interest in the design of the "Kei Hea a Nic?" system, each with different concerns that this design aims to address.

Academic staff (the diary owners) are the professors, lecturers, teaching fellows, and postgraduate researchers who will use the Web Application to manage their own availability, calendar, and contact information. Their main concerns are ease of use with minimal training, fast and reliable updates, and confidence that changes they make will reach the Digital Display quickly and accurately. These concerns are addressed through the Component Diagram (2.3), the Data Flow Diagram (2.4), and the Additional Design Model (2.5), which together show how updates move from the Web Application through to the display with minimal delay.

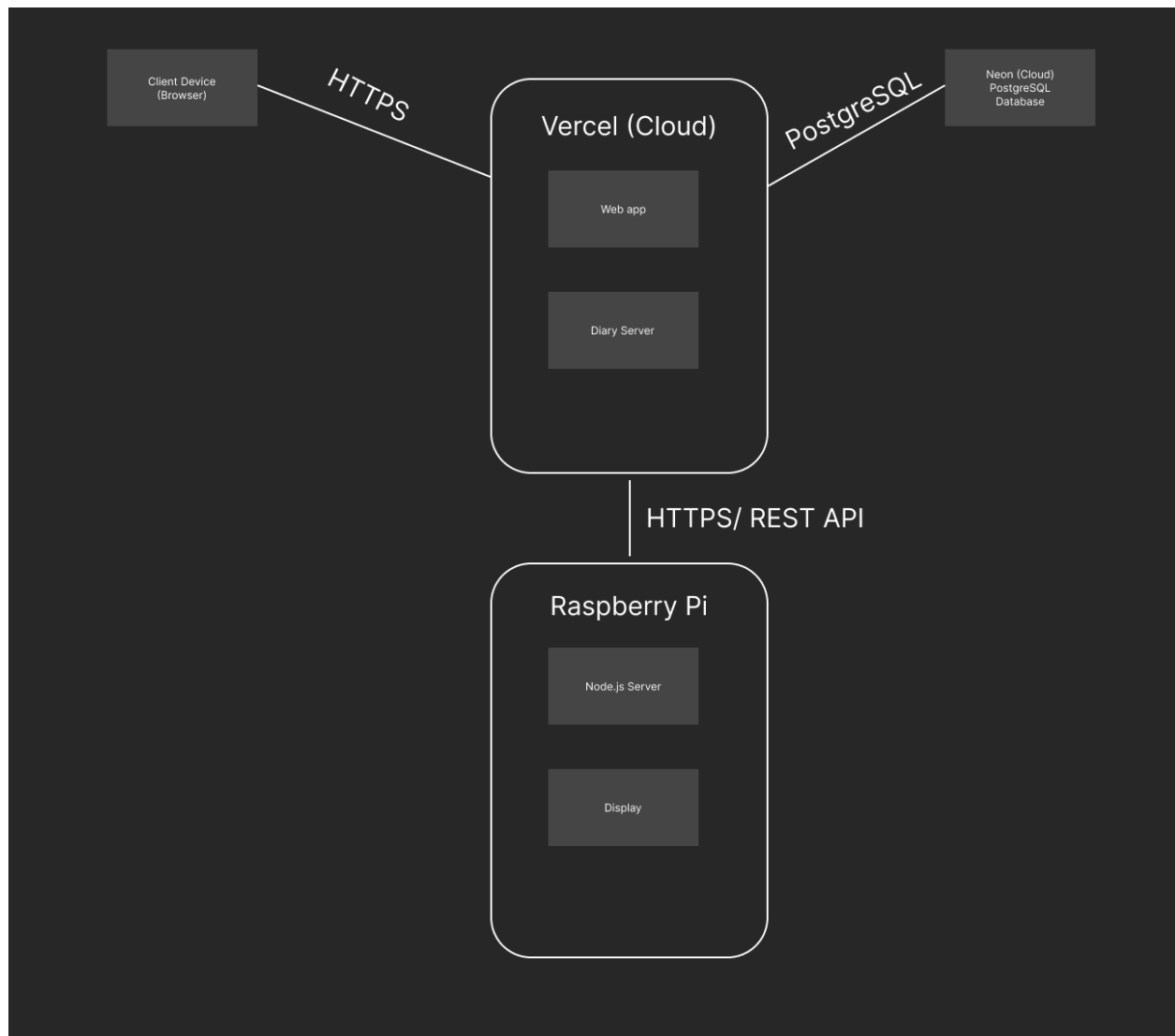
Students, visitors and guests are the people who view the Digital Office Display to check an academic's current availability, timetable, or contact details. Their primary concern is that the information presented is clear, quickly understood from a distance, up to date, and easy to navigate without any prior experience of the system. This is addressed by the Component Diagram (2.3), which defines the Digital Display's three views, and the Data Flow Diagram (2.4), which shows how current information reaches the display.

The client, Nicholas Vanderschantz, representing SCMS academic staff more broadly, is primarily concerned with the feasibility of the prototype, whether the design meets the needs identified during the requirements phase, and the potential for the system to be extended in future (for example, with calendar integrations such as Google Calendar or MyWaikato). These concerns are addressed by the Deployment Diagram (2.2), which shows how the system will be hosted and run, and the Requirements Matrix (2.6), which traces each design element back to the agreed requirements.

The development team is concerned with defining clear component boundaries and producing a design that is realistically achievable within the project timeline and the hardware supplied (the Raspberry Pi and monitor). This is addressed by the Deployment Diagram (2.2) and Component Diagram (2.3), which lay out how responsibilities are divided across the three system components.

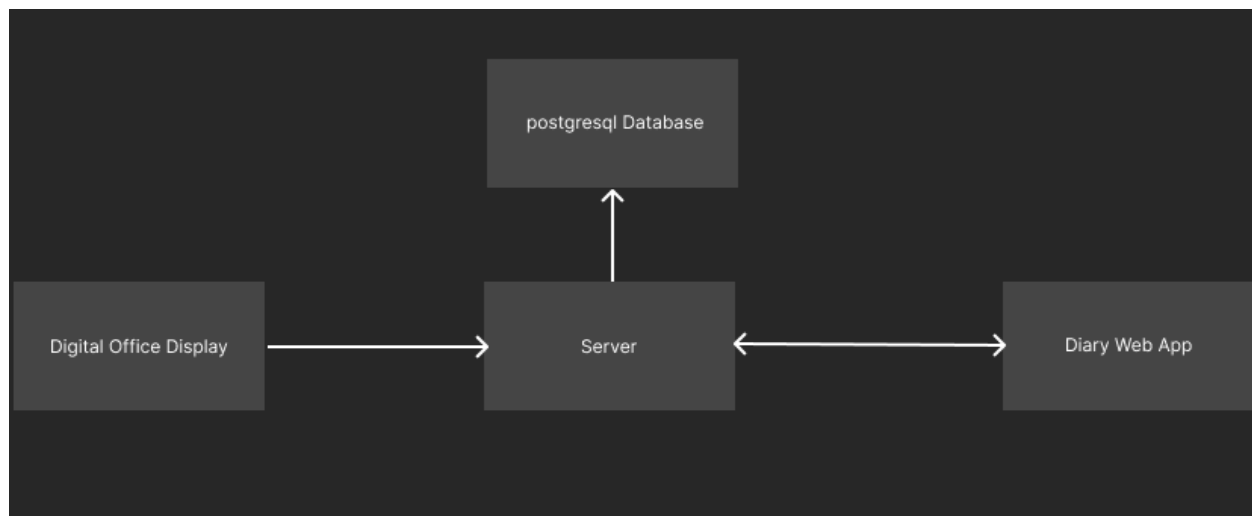
Finally, future maintainers who may extend the system beyond this prototype whether that is moving toward a production-ready deployment or adding features such as external calendar integration are concerned with the maintainability of the system and the clarity of each component's responsibilities. This is addressed by the Component Diagram (2.3) and Deployment Diagram (2.2), which together document how the system is structured and where each part of the functionality resides.

2.2 Deployment Diagram



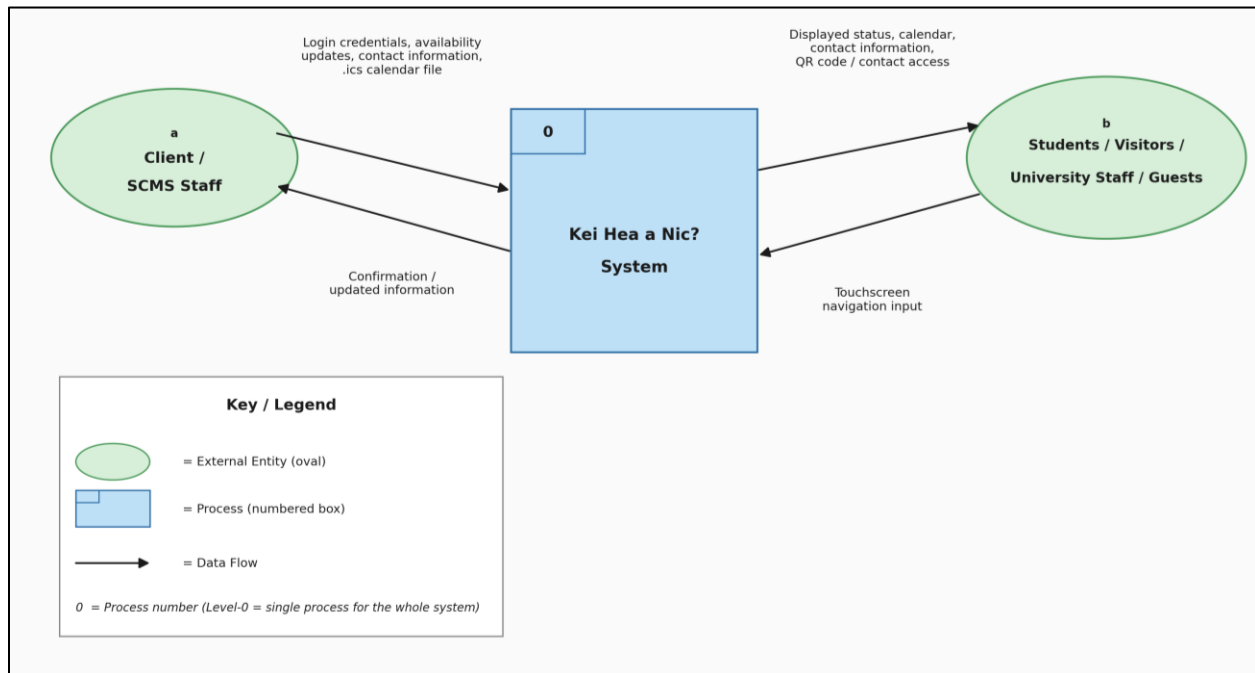
The deployment diagram maps the system's logical components onto the physical hardware and hosting platforms they'll run on, using the box notation for each node. The academic's own device (phone, laptop or some other device that has access to the internet and a viable browser) sits apart from the rest of the system and connects to Vercel over the network Via HTTPS. Vercel is shown as a single cloud node hosting two components, the web app and the diary server's API. The Vercel node is connected to Neon, which acts as the PostgreSQL database, with PostgreSQL used for communication between the application and database. Under the Vercel node is the Raspberry Pi Node that will be displayed outside of the academic office door. The Raspberry Pi has two components, the Node.js server which will run the application that displays the information and also polls the API exposed in the Vercel node to get up to date information. The other component on the Raspberry Pi is the Display that will show the interface. The Raspberry Pi communicates GET information from the Diary Server API over HTTPS using a REST API.

2.3 Component Diagram



The component diagram above breaks the system down into four components and shows how they interact with one another. The Academic Web App acts as a dashboard for academics, allowing them to log in, manage their availability, contact information, and schedule. It communicates with the server to retrieve and update information. The server component sits at the center of the architecture, it manages the information, handling the authentication and processing updates from the web app. The database has one arrow coming in from the server, showing the dependency on the database to store and retrieve information. The last component is the Digital Office Display, which retrieves the latest status from a publicly exposed API from the server component, thus the arrow pointing towards the server.

2.4 Data Flow Diagram



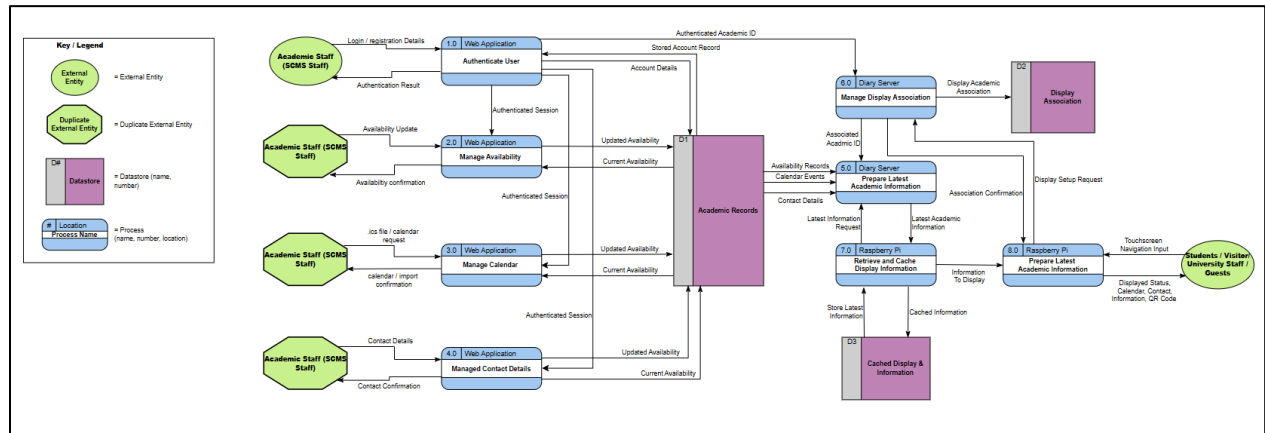
This diagram is the Level 0 context diagram for the "Kei Hea a Nic?" system. Per the definition of a context diagram, the entire software system is represented as a single process, Kei Hea a Nic? System, so that the boundaries of the system can be established before any of its internal detail is considered. Everything that happens inside that boundary, the Academic Web Application, the Diary Server, and the Digital Office Display, along with the database that supports them, is deliberately hidden inside this one process box at this stage. That internal detail is revealed in Level 1 diagrams of data flows.

Two external entities were identified by finding the sources and recipients of each dataflow that lies outside the system. Client/SCMS Staff were identified because they are the source of every piece of information the system stores, and the recipient of confirmation once that information is saved. Students/Visitors/University Staff/Guests were identified because they are the recipient of the information the system displays, and the source of the touchscreen input used to navigate it.

Four dataflows connect these entities to the system. Client/SCMS Staff send in login credentials, availability updates, contact information, and calendar files in .ics format, and receive confirmation or updated information back once a change has been saved. Students, Visitors, University Staff and Guests send touchscreen navigation input into the system as they use the display, and receive the displayed status, calendar, contact information, and QR code back in return.

Only the dataflows that go outside of the system are shown here. Any information that flows entirely within the system, such as the Diary Server storing data in the database or the Digital Display polling the Diary Server for updates, is not shown at this level, since it does not involve an external user entity.

2.5 Level 1 Dataflow Diagram



This diagram expands the single process shown in the Level 0 context diagram (Section 2.4) into the processes, data stores and dataflows that make it up. The two external entities, Academic Staff (SCMS Staff) and Students/Visitor/University Staff/Guests, are carried over unchanged from the Level 0 diagram, since expanding a process never changes the entities outside the system boundary. Academic Staff appears four times in the diagram as a duplicate external entity, once next to each Web Application process it interacts with, so that dataflow lines do not have to cross the whole diagram; each duplicate represents the same entity.

Eight processes are shown, grouped by the component they belong to. Processes 1.0 to 4.0 belong to the Web Application. Process 1.0, Authenticate User, takes in login or registration details from Academic Staff and an account record from the D1 Academic Records data store, and returns an authentication result to the academic and an authenticated session used by the other Web Application processes. Process 2.0, Manage Availability, takes an availability update from the authenticated academic, writes it to D1 and returns the current availability and a confirmation. Process 3.0, Manage Calendar, takes an .ics file or a calendar request from the academic, reads and writes calendar events in D1, and returns the current calendar or an import confirmation. Process 4.0, Manage Contact Details, takes updated contact details from the academic, writes them to D1, and returns the current contact information and a confirmation.

Processes 5.0 and 6.0 belong to the Diary Server. Process 5.0, Prepare Latest Academic Information, takes the availability records, calendar events and contact details held in D1, together with a latest information request from Process 8.0, and returns the latest academic information to the Digital Office Display. Process 6.0, Manage Display Association, takes a display setup request from Process 8.0, reads and writes the association in the D2 Display Association data store, and returns the display's associated academic ID and an association confirmation back to the display.

Processes 7.0 and 8.0 belong to the Digital Office Display and run on the Raspberry Pi. Process 7.0, Retrieve and Cache Display Information, requests the associated academic ID from Process 6.0 and the latest academic information from Process 5.0, stores it in the D3 Cached Display Information data store, and passes the cached information on to Process 8.0. Process 8.0, Prepare Latest Academic Information for Display, takes the cached information from D3 and touchscreen navigation input from visitors, and returns the displayed status, calendar, contact information and QR code that visitors see on the screen. Separating retrieval (7.0) from presentation (8.0) is what allows the display to keep showing the last cached information, with an out-of-date indicator, if the Diary Server cannot be reached.

Three data stores appear at this level, none of which were visible in the Level 0 diagram since they sit entirely inside the system boundary. D1, Academic Records, is the central store maintained by the Diary Server and holds the account, availability, calendar and contact information written by processes 1.0 to 4.0 and read by process 5.0. D2, Display Association, is a small store that records which academic each physical display device is associated with, written and read by process 6.0. D3, Cached Display Information, lives on the Raspberry Pi itself rather than the Diary Server, and holds the most recently retrieved information so the display can keep showing it if the connection to the Diary Server is temporarily lost.

The key/legend in the top-left corner of the diagram identifies each notation used: a green oval for an external entity, a green octagon for a duplicate of an external entity shown elsewhere on the same diagram, a grey-and-purple box labelled with a D-number for a data store, and a blue box labelled with a process number and location for a process. Arrows represent the dataflows connecting these elements, each labelled with the information it carries, consistent with the dataflow notation established in the Level 0 diagram in Section 2.4.

2.6 Requirements Matrix

The functional requirements below are taken from the Product Functions in Section 2.2 of the SRS, grouped by system component and given a stable code (FR-xx). Where the SRS narrows a requirement for the prototype, that has been carried through: the calendar is a single weekly view for the MVP (SRS 3.3), and .ics import is a Should rather than a Must (SRS 3.4).

Academic Web Application

- FR-01: Securely create an account and log in to access only your own data. (Must)
- FR-02: View and update current availability status, expected return time, and custom message. (Must)
- FR-03: View the timetable / calendar in a single simplified weekly view. (Must)
- FR-04: Manage the contact information shown to visitors: email, phone, office location. (Must)
- FR-05: Send all changes to the Diary Server using authenticated requests, propagating to the display automatically. (Must)
- FR-06: Import calendar data using an .ics file. (Should)

Diary Server (Backend / API)

- FR-07: Authenticate users and ensure each academic can only access and modify their own information. (Must)
- FR-08: Store account, availability, calendar, and contact information. (Must)
- FR-09: Provide the latest information to the associated Digital Display and allow regular retrieval. (Must)

Digital Office Display (Raspberry Pi)

- FR-10: Associate the display with a specific academic at setup and remember that association. (Must)
- FR-11: Show current status (with any message), calendar, and contact information across three views. (Must)
- FR-12: Let visitors navigate between the three views using the touchscreen. (Must)
- FR-13: Present a QR code (or equivalent) on the Contact view so visitors can save the details. (Must)
- FR-14: Regularly poll the Diary Server and update automatically when new information is available. (Must)
- FR-15: Run full-screen in kiosk mode and keep showing the last retrieved information if the server is unavailable. (Must)

Cross-Reference Table

The table below maps each functional requirement to the design elements in Sections 2.2 to 2.5 that satisfy it: the system components, the Level-1 DFD elements, and the user interface screens.

Code	Functional Requirement	System Component(s)	Level-1 DFD Element(s)	UI Screen(s)
FR-01	Account creation and secure login	Web App, Diary Server	Login process; credentials dataflow	Login screen
FR-02	View and update availability status	Web App, Diary Server, Database	Update Availability process; data store	Availability screen
FR-03	View weekly calendar / timetable	Web App, Diary Server, Database	Calendar retrieval; data store	Calendar screen
FR-04	Manage contact information	Web App, Diary Server, Database	Contact info dataflow; data store	Contact Information screen
FR-05	Authenticated sync to display	Web App, Diary Server	API request / response dataflows	Confirmation and feedback
FR-06	Import .ics calendar file (Should)	Web App, Diary Server, Database	Import Calendar process; data store	Calendar screen (import control)
FR-07	Authentication and authorisation	Diary Server	Authenticated request dataflow	Backend (no direct screen)
FR-08	Store all academic data	Diary Server, PostgreSQL Database	Data store (Database); write dataflows	Backend (no direct screen)
FR-09	Serve latest data to the display	Diary Server, Raspberry Pi	Provide Latest Information; polling	Backend (feeds display views)

FR-10	Device to academic association	Raspberry Pi, Diary Server	Device association dataflow	Display setup / first-run config
FR-11	Status, Calendar and Contact views	Raspberry Pi and touchscreen	Displayed information dataflow	Status, Calendar, Contact views
FR-12	Touchscreen view navigation	Raspberry Pi and touchscreen	Navigation input dataflow	All three display views
FR-13	QR code for contact details	Raspberry Pi and touchscreen	QR code output dataflow	Contact view (QR element)
FR-14	Poll and auto-update the display	Raspberry Pi, Diary Server	Polling / data request dataflow	All three display views
FR-15	Kiosk mode and offline fallback	Raspberry Pi (kiosk, Node.js)	Cached last-retrieved information	All views + out-of-date notice